COMP2119A/B/C Introduction to Data Structures and Algorithms
Programming Assignment 1 - Course Selection Challenge

Due Date: Nov 4, 2026 7:00pm

IMPORTANT: Please read the following instructions carefully before doing the assignment!

Academic Integrity

- All answers must be your own work. Be aware of academic conduct and copying answers from external sources is strictly prohibited!
- We will use similarity checking software as a test for plagiarism on all the submitted work.

Submission & Grading

Assignment Completion

- You must use C++ to complete the assignment.
- You can use the following libraries: <iostream>, <cstdio>, <string>, <vector>, <set>, <stack>,<queue>, <algorithm>, <math.h>, <limits>, <functional>, <utility>.

  Usage of any library apart from the libraries above will result in 0 marks. Please note that policy-based data structures (under __gnu_pbds::) are not part of the standard library.
- Your program must read from standard input (e.g. scanf / cin) and output the answer to standard output (e.g. print / cout).
  - To read from standard input, you may use: getchar(), scanf(), fgets(), fread(), std::cin, std::getline(). (getchar() is suggested for a large number of int input)
  - To write to standard output, you may use: putchar(), printf(), fputs(), fwrite(), std::cout. (putchar() is suggested for a large number of int output)
- We are unable to assist with debugging your source code due to resource limitations. However, we can provide high-level hints and guidance.

Submission

- Submit your source files through Moodle. You should submit three .cpp source files (without compression) only, one for each task.
- Name your files with the format UniversityNumber-X.cpp, where X represents the task in capital letter (i.e. A, B, C), e.g. 1234554321-A.cpp if your 10-digit UID is 1234554321.
- Marks will be deducted if your submissions are not named correctly.

Judging

- Your program will be judged by a computer automatically. A C++ compiler (of ISO/IEC 14882 standard, i.e. C++14) will be used.
- There will be 10 test cases for each task.
  - Each test case is worth 4 marks for Task A
  - Each test case is worth 3 marks for Task B.
  - Each test case is worth 3 marks for Task C.
- The time limit for each test case is 1 second.
- For each test case, you will receive full marks if your program:
  - Compiles successfully within a reasonable time.
  - Reads from standard input (e.g, scanf / cin).
  - Outputs to standard output (e.g. print / cout).
  - Terminates within the time limit without any errors occurring
  - Has the correct output (including the correct format as specified, with no additional or missing characters).

  If any of the above fails, you will receive 0 marks for that test case.
- Please do not involve randomness and undefined / unspecified / implementation-defined behaviors in your program. Such behaviors include but are not limited to integer overflow, division by zero, and usage of uninitialized variables. You will receive 0 marks for a test case if your program cannot pass it due to these issues.
- If your program tries to modify the grading computer or intervene in the grading process, 0 marks will be given, and your case will be reported immediately. Such behaviors include but are not limited to opening a file, reading from a file (other than standard input), writing to a file (other than standard output), and usage of fork(), exec(), system().
- Parallel computing is not allowed.

Self Testing

- You should test your program by yourself using the provided sample input/output file. The sample input/output is **different** from the ones used to judge your program, and it is designed for you to test your program on your own.
- To test your program on **Linux** or **macOS** (this is the preferred option):
  - Method 1: Run the Python script "test.py" provided
    1. Put your source code "main.cpp", "test.py" at the parent directory of the folder containing test cases for a specific task.
    2. Open a terminal and go to that parent directory by the cd command.
    3. Run the Python script by the terminal command "python3 test.py main.cpp X", where X is the task name (e.g. "python3 test.py main.cpp A" for Task A).

- Method 2: Manual testing
  1. Put your source code "main.cpp" and sample input file "input.txt" in the same directory.
  2. Open a terminal and go to that directory.
  3. Compile your program by "g++ -std=c++14 main.cpp -o main".
  4. Run your program using the command "./main < input.txt > myoutput.txt".
  5. Compare myoutput.txt with sample output file.
  6. Your output in myoutput.txt needs to be **exactly** the same as the sample output. Otherwise, it will be judged as wrong.
  7. You may find the **diff** command useful to help you compare two files, like "diff -w myoutput.txt sampleOutput.txt". The -w option ignores all blanks (SPACE and TAB characters).

- To test your program on **Windows** (with the compiler installed):
  - Method 1: Run the Python script "test.py" provided
    1. Put your source code "main.cpp", "test.py" at the parent directory of the folder containing test cases for a specific task.
    2. Open a terminal and go to that parent directory by the cd command.
    3. Run the Python script by the terminal command "python test.py main.cpp X", where X is the task name (e.g. "python test.py main.cpp A" for Task A).
  - Method 2: Manual testing
    1. Compile your program, and get the executable file, "main.exe".
    2. Put sample input file, "input.txt", in the same directory as "main.exe".
    3. Open command line. Go to the directory that contains files "main.exe" and "input.txt".
    4. Run "main.exe < input.txt > myoutput.txt".
    5. Compare myoutput.txt with sample output file. You may find "fc.exe" tool useful.
    6. Your output in myoutput.txt needs to be **exactly** the same as the sample output. Otherwise, it will be judged as wrong.

- You should also use the online compiler below to check your work (in C++ 14): `https://www.onlinegdb.com/online_c++_compiler`

- The scores you get in self-testing do not represent your final grade. The final score is determined by the machine we used to grade your work. You cannot appeal your grade by using the results obtained in self-testing as evidence.

Manual Code Checking

If your code does not compile or work properly during grading, we will check your source code manually **ONLY** if it passes the following online compiler (in C++ 14):

`https://www.onlinegdb.com/online_c++_compiler`

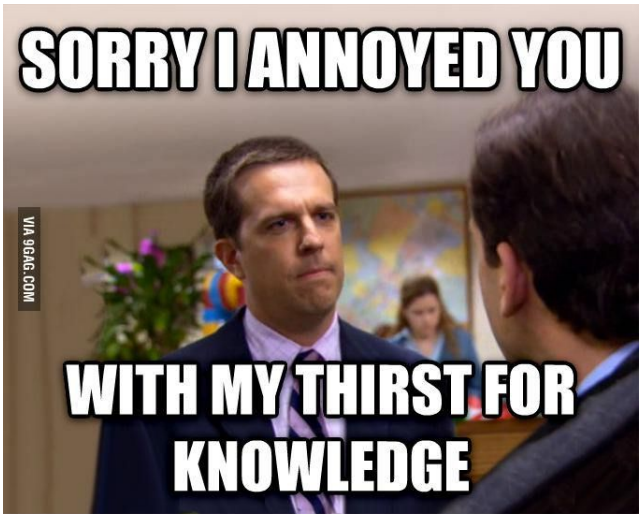
Therefore, students must ensure that their code compiles properly using the online test, otherwise they may receive 0 points.

Background & Important Definitions

Computer Science knowledge base grows exponentially under the AI era. In the near future of 2050, there are thousands of COMP courses offered by the HKU School of CDS every year. Some courses are prerequisites of one another, forming a complex network of courses.

Handsome Jeff has just been accepted into the HKU School of CDS as a new Computer Science major student. During his studies, he needs to pass different courses in the course network. He investigates the course network in detail and comes to you, bringing tons of questions on how to plan his study path wisely.

Complete the tasks in this assignment to answer his questions in a smart and efficient way!



Here are some important definitions and rules that apply to all the tasks:

- In each task, you are given a set of n courses and m relationships. Each relationship $A \to B$ represents course A is a prerequisite of course B.
- You may assume that the relationships do not form a cycle (i.e. Relationship chains like $A \to B \to C \to A$ do not exist).
- To study a course, the following eligibility rules apply:
  - If a course does not have a prerequisite, a student is eligible to study it anytime.
  - If a course has only one prerequisite course, a student must pass that prerequisite course to become eligible for studying that course.
  - If a course has multiple prerequisite courses, a student only needs to pass at least one of them to become eligible for studying that course.
- A student cannot study a course again if he/she has passed it already.
- **At the beginning of each task, Handsome Jeff is a new student and he has passed 0 courses (i.e. Advanced standing is not applicable).** This means he can only choose to study courses that do not have a prerequisite at the start.

Task A [40%]
Handsome Jeff eagerly wants to study course s. He wants to obtain the eligibility to study course s by traversing the easiest study path.

To find such a path, Jeff has assigned a difficulty level c_i for each course i. Define the total difficulty level of a study path as the sum of difficulty level for all courses on the study path. The target is to find the lowest possible total difficulty level of a study path, such that Jeff becomes eligible to study course s after passing all the courses on the path.

Complete the file **UniversityNumber-A.cpp** to calculate the answer.

Input Format:

- The first line has two integers n and m. n denotes the number of courses available, m denotes the number of unique prerequisite relationships between the courses.
- The next line contains n integers, where the $(i+1)^{th}$ integer c_i represents the difficulty level of course i.
- Each of the next m lines contains two integers a, b, where a and b are distinct course ids. Course a is a prerequisite of course b.
- The last line contains an integer s, which denotes the course id Jeff eagerly wants to study.

For all the input data, $1 \leq n \leq 100000$; $0 \leq m \leq 300000$; $1 \leq c_i \leq 10$; $0 \leq a, b, s \leq n-1$. You can also assume that the course ids must start from 0 and end at $n-1$.

Output Format:
Output a single integer *total*, the lowest possible total difficulty level of a study path that allows Jeff to become eligible to study course s.

Sample Test Cases:

Sample Input	Sample Output	Explanation
10 13 8 1 3 2 4 2 5 8 9 7 0 1 1 5 1 6 2 1 3 0 3 4 4 2 4 5 5 6 5 9 7 2 7 4 8 7 1	9	To make Jeff eligible to study course 1 with a study path having the lowest total difficulty level, take transitions [3 4], then [4 2], then [2 1]. He needs to pass course 3, 4, 2 to become eligible to study course 1. Therefore, the lowest total difficulty level is 2 + 4 + 3 = 9.
6 10 2 1 3 6 2 1 0 1 0 2 0 3 1 2 1 3 1 4 2 3 2 4 3 4 4 5 0	0	There are no prerequisites for course 0. Jeff is eligible to study it at time 0. Therefore, the program should output 0.
5 3 2 4 1 2 7 0 1 1 2 0 2 4	0	There are no prerequisites for course 4. Jeff is eligible to study it at time 0. Therefore, the program should output 0.

Hint:

- A reasonable algorithm should have $O((n+m)\log n)$ time complexity.
- 50% of the test cases have input size $1 \leq n \leq 1000$ and $0 \leq m \leq 1000$, while the other 50% of the test cases have the full range of input size specified.

Task B [30%]
Ben, the academic advisor of Jeff, has reminded him of the importance of passing course s during the Orientation Day. He told Jeff that if he cannot pass course s, it may become impossible for him to study many other courses.

Jeff wants to verify his claim by counting the number of other courses (i.e. excluding course s) that are no longer reachable (i.e. unable to obtain eligibility to study) from any course with no prerequisites when course s cannot be passed.

Complete the file **UniversityNumber-B.cpp** to calculate the answer.

Input Format:

- The first line has two integers n and m. n denotes the number of courses available, m denotes the number of unique prerequisite relationships between the courses.
- Each of the next m lines contains two integers a, b, where a and b are distinct course ids. Course a is a prerequisite of course b.
- The last line contains an integer s, which denotes the course id Ben has mentioned.

For all the input data, $1 \leq n \leq 100000$; $0 \leq m \leq 300000$; $0 \leq a, b, s \leq n - 1$. You can also assume that the course ids must start from 0 and end at $n - 1$.

Output Format:
Output an integer *course*, the number of courses (excluding course s) that are no longer reachable from any course with no prerequisites when course s cannot be passed.

Sample Test Cases:

Sample Input	**Sample Output**	**Explanation**
9 11 0 1 1 5 1 6 3 0 3 4 4 2 4 5 5 6 7 2 7 4 8 7 3	2	Course 0, 1 can be studied only if Jeff passes course 3. Therefore, the program should output 2.
6 10 0 1 0 2 0 3 1 2 1 3 1 4 2 3 2 4 3 4 4 5 2	0	There are no courses that can be studied only if Jeff passes course 2. Therefore, the program should output 0.

Hint:

- A reasonable algorithm should have $O(n + m)$ time complexity.
- 50% of the test cases have input size $1 \leq n \leq 1000$ and $0 \leq m \leq 1000$, while the other 50% of the test cases have the full range of input size specified.

Task C [30%]
Handsome Jeff finds the course network too complicated to understand. He wants to arrange an ordering to all courses, such that if there is a prerequisite relationship $A \to B$, course A must appear before course B in the order. After constructing this order, Jeff assigns an ordering number to each course. The first course in the order has ordering number 1, the second course has ordering number 2, and so on.

To make the order unique and easier to check, Jeff decides that whenever there are ties, the course with the smaller course id must be placed first.

For a given course s, Jeff wants to know the ordering number assigned to it.

Complete the file **UniversityNumber-C.cpp** to calculate the answer.

Input Format:

- The first line has two integers n and m. n denotes the number of courses available, m denotes the number of unique prerequisite relationships between the courses.
- Each of the next m lines contains two integers a, b, where a and b are distinct course ids. Course a is a prerequisite of course b.
- The last line contains an integer s, which denotes the target course id.

For all the input data, $1 \leq n \leq 100000$; $0 \leq m \leq 300000$; $0 \leq a, b, s \leq n - 1$. You can also assume that the course ids must start from 0 and end at $n - 1$.

Output Format:
Output an integer *order*, the ordering number assigned to course s.

Sample Test Cases:

Sample Input	Sample Output	Explanation
8 10 0 1 1 5 1 6 2 1 3 0 3 4 4 2 4 5 5 6 7 3 4	4	Course 7 has no prerequisites. It has ordering number = 1. Course 3 has only course 7 as its prerequisite. It has ordering number = 2. Course 0 and 4 have only course 3 as their prerequisite. Since course 4 has a larger id than course 0, course 0 has ordering number = 3 and course 4 has ordering number = 4. Therefore, the program should output 4.
6 10 0 1 0 2 0 3 1 2 1 3 1 4 2 3 2 4 3 4 4 5 4	5	The ordering number for course 0, 1, 2, 3, 4, 5 are 1, 2, 3, 4, 5, 6 respectively. Therefore, the program should output 5.

Hint:

- A reasonable algorithm should have $O((n+m)\log n)$ time complexity.
- 50% of the test cases have input size $1 \leq n \leq 1000$ and $0 \leq m \leq 1000$, while the other 50% of the test cases have the full range of input size specified.

Enquiries

If you have any enquiries related to the assignment, please contact the TA below for help:

- Question Design: Jeff Siu (Email: jeffsiu1@hku.hk)
- Grading: Anqi Zhang (Email: anqizhang@connect.hku.hk)